

Start coding or [generate](#) with AI.

Mount Drive, verify GPU

```
from google.colab import drive
drive.mount('/content/drive')

import tensorflow as tf
print("GPU:", tf.config.list_physical_devices('GPU'))
```

```
Mounted at /content/drive
GPU: []
```

Extract banana dataset

```
import zipfile, os
os.makedirs('/content/dataset', exist_ok=True)
with zipfile.ZipFile('/content/drive/MyDrive/Banana_Disease_Project_Dataset.zip', 'r') as z:
    z.extractall('/content/dataset')
```

Kaggle setup + download 4 negative sources

```
from google.colab import files
files.upload() # kaggle.json

!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle --quiet

sources = [
    "prasunroy/natural-images",
    "puneet6060/intel-image-classification",
    "gpiosenka/100-bird-species",
    "csafrit2/plant-leaves-for-image-classification",
]
for src in sources:
    !kaggle datasets download -d {src} -p /content/negative_raw --unzip --quiet
print("Downloads complete")
```

Choose files kaggle.json

kaggle.json(application/json) - 66 bytes, last modified: 23/08/2026 - 100% done

Saving kaggle.json to kaggle.json

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest version (2.2.2)

Dataset URL: <https://www.kaggle.com/datasets/prasunroy/natural-images>

License(s): CC-BY-NC-SA-4.0

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest version (2.2.2)

Dataset URL: <https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

License(s): copyright-authors

403 Client Error: Forbidden for url: <https://api.kaggle.com/v1/datasets.DatasetApiService/GetDatasetMetadata>

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest version (2.2.2)

Dataset URL: <https://www.kaggle.com/datasets/csafrut2/plant-leaves-for-image-classification>

License(s): Community Data License Agreement - Sharing - Version 1.0

Downloads complete

Build final 4-class dataset

```
import shutil, random

FINAL_DIR = '/content/final_dataset'
for cls in ['Black_Sigatoka', 'Fusarium_Wilt', 'Healthy', 'Not_Banana_Leaf']:
    os.makedirs(f'{FINAL_DIR}/{cls}', exist_ok=True)

DISEASE_SRC = '/content/dataset/Banana_Disease_Project_Dataset/Dataset'
for cls in ['Black_Sigatoka', 'Fusarium_Wilt', 'Healthy']:
    for fname in os.listdir(f'{DISEASE_SRC}/{cls}'):
        shutil.copy(f'{DISEASE_SRC}/{cls}/{fname}', f'{FINAL_DIR}/{cls}/{fname}')

random.seed(42)
IMAGE_EXTS = {".jpg", ".jpeg", ".png"}
PER_SOURCE_CAP = 3000

count = 0
for d in os.listdir('/content/negative_raw'):
    d_path = os.path.join('/content/negative_raw', d)
    if not os.path.isdir(d_path):
        continue
    imgs = [os.path.join(r, f) for r, _, fs in os.walk(d_path)
            for f in fs if os.path.splitext(f)[1].lower() in IMAGE_EXTS]
    random.shuffle(imgs)
    for path in imgs[:PER_SOURCE_CAP]:
        shutil.copy(path, f'{FINAL_DIR}/Not_Banana_Leaf/neg_{count}.jpg')
        count += 1

for cls in ['Black_Sigatoka', 'Fusarium_Wilt', 'Healthy', 'Not_Banana_Leaf']:
    print(cls, ":", len(os.listdir(f'{FINAL_DIR}/{cls}')))
```

```
Black_Sigatoka : 3496
Fusarium_Wilt : 4932
Healthy : 3339
Not_Banana_Leaf : 18000
```

```
import os, random
```

```
NOT_BANANA_DIR = f'{FINAL_DIR}/Not_Banana_Leaf'
```

```

TARGET_COUNT = 5000

all_files = os.listdir(NOT_BANANA_DIR)
print(f"Current count: {len(all_files)}")

if len(all_files) > TARGET_COUNT:
    random.seed(42)
    random.shuffle(all_files)
    to_remove = all_files[TARGET_COUNT:]
    for fname in to_remove:
        os.remove(os.path.join(NOT_BANANA_DIR, fname))

print(f"New count: {len(os.listdir(NOT_BANANA_DIR))}")

```

```

Current count: 18000
New count: 5000

```

Clean corrupted images

```

from PIL import Image

def clean_folder(folder):
    removed = 0
    for fname in os.listdir(folder):
        fpath = os.path.join(folder, fname)
        try:
            Image.open(fpath).verify()
        except Exception:
            os.remove(fpath); removed += 1
    print(f"{folder}: removed {removed}")

for cls in ['Black_Sigatoka', 'Fusarium_Wilt', 'Healthy', 'Not_Banana_Leaf']:
    clean_folder(f'{FINAL_DIR}/{cls}')

```

```

/content/final_dataset/Black_Sigatoka: removed 0
/content/final_dataset/Fusarium_Wilt: removed 0
/content/final_dataset/Healthy: removed 0
/content/final_dataset/Not_Banana_Leaf: removed 0

```

Save consolidated dataset to Drive (critical — protects against Colab disconnects)

```

import os
import shutil

# Specific folders to delete to free up disk space
folders_to_delete = ["dataset", "negative_raw"]

for folder in folders_to_delete:
    if os.path.exists(folder):
        shutil.rmtree(folder)
        print(f"Deleted: {folder}")
    else:
        print(f"Folder not found: {folder}")

```

```
print("\nCleanup finished! 'final_dataset' and 'drive' remain untouched.")
```

```
Deleted: dataset  
Deleted: negative_raw
```

```
Cleanup finished! 'final_dataset' and 'drive' remain untouched.
```

```
print("Zipping final dataset – avoids re-downloading from Kaggle in future sessions...")  
shutil.make_archive('/content/drive/MyDrive/final_dataset', 'zip', FINAL_DIR)  
print("Saved: /content/drive/MyDrive/final_dataset.zip")
```

```
Zipping final dataset – avoids re-downloading from Kaggle in future sessions...  
Saved: /content/drive/MyDrive/final_dataset.zip
```

Build tf.data pipeline with augmentation

```
from tensorflow.keras import layers  
  
IMG_SIZE, BATCH_SIZE, SEED = (224, 224), 32, 42  
  
train_ds = tf.keras.utils.image_dataset_from_directory(  
    FINAL_DIR, validation_split=0.2, subset="training", seed=SEED,  
    image_size=IMG_SIZE, batch_size=BATCH_SIZE)  
val_ds = tf.keras.utils.image_dataset_from_directory(  
    FINAL_DIR, validation_split=0.2, subset="validation", seed=SEED,  
    image_size=IMG_SIZE, batch_size=BATCH_SIZE)  
  
class_names = train_ds.class_names  
print("Classes:", class_names) # note this order – needed in backend later  
  
AUTOTUNE = tf.data.AUTOTUNE  
train_ds = train_ds.prefetch(AUTOTUNE)  
val_ds = val_ds.prefetch(AUTOTUNE)  
  
class RandomGaussianBlur(layers.Layer):  
    """Applies a real 3x3 Gaussian blur to ~prob fraction of batches during training."""  
    def __init__(self, prob=0.3, **kwargs):  
        super().__init__(**kwargs)  
        self.prob = prob  
  
    def call(self, images, training=None):  
        if not training:  
            return images  
        apply_blur = tf.random.uniform([]) < self.prob  
        def blur():  
            kernel = tf.constant([[1,2,1],[2,4,2],[1,2,1]], dtype=tf.float32) / 16.0  
            kernel = tf.reshape(kernel, [3,3,1,1])  
            kernel = tf.tile(kernel, [1,1,3,1])  
            return tf.nn.depthwise_conv2d(images, kernel, strides=[1,1,1,1], padding='SAME')  
        return tf.cond(apply_blur, blur, lambda: images)  
  
data_aug = tf.keras.Sequential([  
    layers.RandomFlip("horizontal"),  
    layers.RandomRotation(0.15),
```

```

layers.RandomZoom(0.15),
layers.RandomBrightness(0.25), # lighting variation
layers.RandomContrast(0.25), # over/underexposed conditions
RandomGaussianBlur(prob=0.3), # real blur – simulates out-of-focus phone photos
])

```

```

Found 16767 files belonging to 4 classes.
Using 13414 files for training.
Found 16767 files belonging to 4 classes.
Using 3353 files for validation.
Classes: ['Black_Sigatoka', 'Fusarium_Wilt', 'Healthy', 'Not_Banana_Leaf']

```

Build and train baseline (frozen base)

```

base_model = tf.keras.applications.MobileNetV2(input_shape=(*IMG_SIZE, 3), include_top=False, weights='imagenet')
base_model.trainable = False

inputs = tf.keras.Input(shape=(*IMG_SIZE, 3))
x = data_aug(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)
x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(len(class_names), activation='softmax')(x)

model = tf.keras.Model(inputs, outputs)
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

callbacks = [
    tf.keras.callbacks.ModelCheckpoint(
        '/content/drive/MyDrive/checkpoints/unified_baseline.keras',
        save_best_only=False, save_freq='epoch')
]

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5
9406464/9406464 — 0s 0us/step

```

from sklearn.utils.class_weight import compute_class_weight
import numpy as np

y_train = np.concatenate([y for x, y in train_ds], axis=0)
class_weights = compute_class_weight('balanced', classes=np.unique(y_train), y=y_train)
class_weight_dict = dict(enumerate(class_weights))
print(class_weight_dict)

history = model.fit(train_ds, validation_data=val_ds, epochs=10,
                    callbacks=callbacks, class_weight=class_weight_dict)

```

```

{0: np.float64(1.1917199715707179), 1: np.float64(0.8455622793746849), 2: np.float64(1.2697841726618706), 3: np.float64(0.8398447282744803)}
Epoch 1/10
420/420 — 4326s 10s/step - accuracy: 0.8648 - loss: 0.3480 - val_accuracy: 0.9514 - val_loss: 0.1387
Epoch 2/10
420/420 — 2662s 6s/step - accuracy: 0.9302 - loss: 0.1817 - val_accuracy: 0.9645 - val_loss: 0.1026
Epoch 3/10
420/420 — 2350s 6s/step - accuracy: 0.9402 - loss: 0.1529 - val_accuracy: 0.9597 - val_loss: 0.0991

```

Epoch 4/10
420/420 ————— 2368s 6s/step - accuracy: 0.9497 - loss: 0.1413 - val_accuracy: 0.9654 - val_loss: 0.0930
Epoch 5/10
420/420 ————— 2361s 6s/step - accuracy: 0.9460 - loss: 0.1399 - val_accuracy: 0.9687 - val_loss: 0.0841
Epoch 6/10
420/420 ————— 2442s 6s/step - accuracy: 0.9478 - loss: 0.1427 - val_accuracy: 0.9669 - val_loss: 0.0833
Epoch 7/10
420/420 ————— 2428s 6s/step - accuracy: 0.9468 - loss: 0.1391 - val_accuracy: 0.9663 - val_loss: 0.0864
Epoch 8/10
420/420 ————— 2381s 6s/step - accuracy: 0.9506 - loss: 0.1319 - val_accuracy: 0.9663 - val_loss: 0.0835
Epoch 9/10
420/420 ————— 2400s 6s/step - accuracy: 0.9448 - loss: 0.1433 - val_accuracy: 0.9663 - val_loss: 0.0823
Epoch 10/10
420/420 ————— 2491s 6s/step - accuracy: 0.9471 - loss: 0.1391 - val_accuracy: 0.9663 - val_loss: 0.0841

```
model.save('/content/drive/MyDrive/checkpoints/final_best_model.keras')
```

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.